IMPERIAL

MEng Individual Project

Imperial College London

Department of Computing

Tools for Foundational Verification of Probabilistic Programs

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
Prof. Nicolas Wu

May 30, 2026

Contents

1 Overview **4**
- 1.1 Contributions . . . 4
- 1.2 Motivation . . . 5
  - 1.2.1 Discrete vs Continuous distributions . . . 5
  - 1.2.2 Probabilistic programs in the wild . . . 5
- 1.3 Lean and related formalisation projects . . . 5

2 Background **7**
  - 2.0.1 Monads . . . 7
  - 2.0.2 Type Theory . . . 7
- 2.1 Measure Theory . . . 7
  - 2.1.1 Borel spaces and lebesgue measures . . . 8
  - 2.1.2 Product of measurable spaces . . . 8
- 2.2 Semantics of Probabilistic Programs . . . 8
  - 2.2.1 Types in BPPL . . . 11
  - 2.2.2 The let binding problem . . . 11
  - 2.2.3 Semantics of Let binding . . . 12
- 2.3 Separation Logic . . . 13
  - 2.3.1 Classical separation logic . . . 13
  - 2.3.2 Zoo of separation logics . . . 13
  - 2.3.3 Iris and MoSeL . . . 16
- 2.4 Theorem Proving . . . 16
  - 2.4.1 Dependent type theory . . . 16
  - 2.4.2 Metaprogramming . . . 19

3 Formalisation of Lilac **21**
- 3.1 Syntax and semantics of Appl . . . 21
- 3.2 Semantics of Lilac propositions . . . 22
  - 3.2.1 Reviewing Probabilistic Separation Logics . . . 22
  - 3.2.2 Lilac . . . 23
- 3.3 Instantiating Iris . . . 23
- 3.4 Pi-lambda-theorem as an induction principle . . . 24
- 3.5 Binder Discipline . . . 25
  - 3.5.1 Dependent De Brujin Indices . . . 25
  - 3.5.2 PHOAS . . . 25
- 3.6 Instantiating Iris . . . 25
  - 3.6.1 Deep vs Shallow Embeddings . . . 26
  - 3.6.2 BI in Iris . . . 26
  - 3.6.3 First Attempt: Deeply embedded logic instantiation . . . 27
  - 3.6.4 Shallow embedding of the logic . . . 27
- 3.7 Deparametrisation of LProp . . . 27
- 3.8 Weakest precondition . . . 28
- 3.9 Deparametrisation of LProp . . . 28
- 3.10 Variables . . . 28
  - 3.10.1 Substitution inconsistency . . . 29
- 3.11 Equality of dependently typed records in Lean . . . 29

3.12 Choice of Kripke Resource Monoid (KRM) over Unital ordered Resource algebra (ORA) . . . 29
3.13 Deep vs shallow embedding . . . 29
3.14 Support for recursion over the data structure even with (shallow?/deep? see SSProve talk) embedding . . . 29
3.15 Strong abstractions across the proof development . . . 29
3.15.1 The language . . . 29
3.15.2 Resource monoid . . . 29
3.15.3 Separate treatment of BI connectives and Probability specific connectives . 29
3.16 Metaprogramming and tactical support . . . 29

Chapter 1

Overview

1.1 Contributions

- We provide the first complete implementation of a probabilistic separation logic. Specifically Lilac is formalised in the Lean theorem prover complete with both soundness proof and extensive support for tactic-based weakest-precondition style reasoning about programs
- One of the first non-trivial instantiation of the Lean port of the Iris framework (originally developed in Rocq).
- A Novel treatment of infinite looping programs which may construct a convergent sequence of values. The correctness of some algorithms for probabilistic inference is given by convergence of such a sequence. So add a connective to Lilac for reasoning about infinite loops and re-establish the soundness of the logic.

  "The search for well-specified probabilistic models—models that correctly describe the processes that produce data—is one of the enduring ideals of the statistical sciences. Yet, in only the simplest of settings are we able to achieve this goal" - Papamakarios et al

  "Although our tactics perform symbolic execution in small steps, one can easily define a tactic that performs symbolic execution repeatedly. However, in concurrent separation logic, small step symbolic execution is often needed so as to be able to open invariants around atomic expressions" - Krebbers et al. 2017b https://iris-project.org/pdfs/2017-popl-proofmode-final.pdf

  so it makes sense for be to automatically apply wp_let, wp_ret and wp_unif and wp_ber immediately. However is it not possible for this to get stuck if we are not dealing with unif and ber alone (eg. complicate (det/rand)val coming from the meta logic??)

  Shall I talk about logic programming via type classes? Probably briefly should do. To illustrate why and how certain type classes were implemented for certain propositions. Especially interesting is this line [Krebbers et al. 2017b] "However, since we use a shallow embedding to represent the propositions of our object logic, propositions are semantic objects, so we cannot just write a Coq function to perform this traversal. Instead, we use logic programming using type classes to perform such manipulations on the meta level"

  This work is a good example exhibiting an easy to use separation logic proofmode, without the complexity of step-indexing (as in most iris projects). This is because our language is first-order and does not support recursion and yet has an orthogonal axis of intricacy which makes it interesting. As such it makes a good case study for the application of modern proofmode construction techniques decoupled from the step-indexed world of Iris in which practically every other significant program logic lives.

  Expressive tactic-based reasoning about programs is developed through Lean meta-programming.

1.2 Motivation

Here we give an informal account of what probabilistic programs are and why we would care to use them A Probabilistic programming language is not an established or general paradigm of programming such as Functional or Imperative programming. Most generally probabilistic programs are simply programs which have a `sample` construct for sampling from probability distributions, and optionally, an `observe` for specifying a new distribution given an old distribution and some newly observed data.

Statisticians use languages such as Stan Carpenter et al. [2017], Hakaru Narayanan et al. [2016] and Anglican Wood et al. [2014], to build complex statistical models succinctly using the expressivity of a programming language. Note that here, a model is simply a probability distribution over some quantity of interest, take for instance the probability of a 3d structure being correct for the given protein sequence, (other examples may come from weather forecasting, medical diagnosis, economics, statistical mechanics etc). "Running" this code corresponds to running a simulation of the model, where we can get answers such as the mean or variance, or just samples (sample protein structure to test in a lab in the example).

What distinguishes a probabilistic programming language is that it decouples the specification of a statistical model from the algorithm for its simulation (henceforth called probabilistic inference). Inference algorithms are often highly optimised and specialised to the specific model and the interpreter of a PPLs attempts to make the best choice/combination of inference algorithms for the given code (model), relieving the user of this additional work.

So in other words PPLs are a layer of abstraction which sits between a statistical model and the inference algorithms which simulate it. This abstraction unlocks the verification of correctness properties of the statistical model, which led to the development of program logics for reasoning about PPLs. This will be the main topic of this report.

1.2.1 Discrete vs Continuous distributions

First let us define what discrete and general (continuous) PPLs are. A discrete PPL does not allow specifying a probability distribution over $\mathbb{R}$. This is achieved most straightforwardly by not including a type corresponding to $\mathbb{R}$ in the syntax of a PPL. A general PPL will have a type corresponding to $\mathbb{R}$. This seemingly small detail has significant consequences on the semantics of a PPL which in turn dictates the complexity of program logics for such a language. This dichotomy largely exists due to difficulties with semantics of continuous PPLs. Take for example that a semantics for higher order PPLs was given by [Staton, 2017] within the decade.

However, all the PPLs above are continuous. Indeed PPLs are made only relevant when working with continuous distributions, as the remove the difficulty of working with the probabilistic inference algorithms. Much less sophistication is required in the inference algorithms for discrete algorithms, so discrete PPLs are not of much interest. So why are programs logics for discrete PPLs such as [Bao et al., 2025] and Lohse et al. [2026] important?

1.2.2 Probabilistic programs in the wild

In the most general sense a probabilistic program is just any program which samples from a distribution. Support for this can be found in the standard libraries of every major programming language, such as Python, C++ or Java. Some of critical applications manipulating discrete distributions, where formal verification is desirable is Cryptography and differential privacy . In fact one of the first major applications of probabilistic *separation* logic is being pursued in the formalisation of [Bao et al., 2025] to prove soundness of Zero Knowledge Proof Systems (subfield of Cryptography).

1.3 Lean and related formalisation projects

Foundational verification, consists of machine checked proofs of theorems, assuming only the axioms of mathematics. Here we are concerned with verification of software, so the theorems in question express properties of the software. The axioms of mathematics that we use is a dependent type

theory. In this setting of dependent type theory, checking the correctness of proof is the same thing as checking if a proof (program) is well typed, due to the Curry-Howard Isomorphism. Further the tool we use to for machine-checked proofs is an interactive theorem prover with support for proofs using "tactics". Tactical support is just metaprogramming, trying to encapsulate reasoning principles a person would use when writing pen-and-paper proofs. There are two proof assistants that were strongly considered for the purposes of this project: Lean and Rocq, former having proofs of required theorems in measure theory and the latter having the original implementation of the Iris framework. Lean was chosen since the front-end of the Iris proof mode (MoSeL, see 2.3.3) had been fully ported to Lean very recently.

There has been an exciting amount of recent activity in formalisations related to this work, which has enabled the formalisation of Lilac (see, 3.2.2) in Lean to be a realistic goal. Most importantly, the formalisation of the disintegration theorem in Degenne [2025] is only available in Lean at the moment. Disintegration theorem also involved the formalisation of kernels which are actually available in several other proof assistants. Quasi-borel spaces Heunen et al. [2017] (in which higher order probabilistic programs are interpreted) have been formalised in Isabelle/HOL in Hirata et al. [2023], along with kernels. s-finite-kernels can also be found in rocq Affeldt et al. [2025].

A lot of background in Lean was required to undertake this project. For example, we use the full expressivity of inductive types and inductive families. Also an understanding of propositional vs definitional equality which is related to Proof irrelevance Avigad et al. [2024] and how that's implemented in Lean's type theory by special treatment of the lowest Universe Carneiro [2019], is very helpful.

Chapter 2

Background

Probabilistic programming is a paradigm where we are defining statistical models using code. It is a means of describing complex distributions over some space in a structured way. Probabilistic programming language (PPL) also come with an interpreter (probabilistic inference engine) which allows the programmer to sample a term from the distribution denoted by a program and indeed sample distributions while constructing new distributions.

More interestingly a Bayesian probabilistic programming languages (BPPL) also allows one to "observe" an event and update their belief of the world (update the probability distribution they associate with some ocurrence). Here we take the bayesian perspective underpinned by Baye's law which describes how we update a probability distribution based

A simple example taken from Staton [2017] is reproduced below to show how

2.0.1 Monads

The reader is assumed to have had been exposed to a monadic style of programming (for example Haskell). Mathematical/categorical perspective is not strictly necessary, and is thus omitted.

Here we give a brief refresher on the computational perspective of monads It is relevant to understand the semantics of probabilistic programming (Giry Monad) and Leans metaprogramming library (MetaM monad).

Nevertheless the categorical/mathematical perspective on monads though not strictly necessary to understand this work, it's a valuable alternative perspective. For example the justification for the semantics of higher order probabilistic programs being an open problem till 2017, if given naturally in categorical language (the category of measurable spaces in which types are interpreted is not Cartesian closed).

From the computational perspective, A monad can be understood as a map from type A to M A. It is additionally associated with two function Ret and bind, with the following your signatures. Monads are actually monoids with Ret acting as unit and bind acting as the binary operation. The laws that Ret and bind obey are as follows which should remind one of the monoid laws.

2.0.2 Type Theory

Readers are also assumed to have some familiarity with type theory, in particular having seen inference rules.

2.1 Measure Theory

In order to understand the semantics of a probabilistic programming, a formal definition for probability is needed. Measure theory underpins probability theory and an overview is given here. We want to be able to talk about the probabilities of some event occurring, which is modelled by a `random variable`. In order to define a random variable we first need to define Measures and

Measurable Space. A measurable space is a pair (Ω, Σ_Ω), where Ω is some Set, and $\Sigma_\Omega \subseteq \mathcal{P}(\Omega)$ is called the σ-algebra. We interpret each element of Σ_Ω to be an event (such as a coin flip landing heads). We would like the σ-algebra to be closed under complements and countable unions. From the presentation so far, it may seem like all singleton sets must be events but due to technical reasons (see Axler [2020] Ch. 2), this cannot be in certain cases where Σ is infinite.

A measure on a measurable space is a function $\mu : \Sigma_\Omega \to [0, \infty]$.

A probability measure on the other hand is more restrictive: $\mu : \Sigma_\Omega \to [0, 1]$. It gives the probability of an event $E \in \Sigma\_\Omega$ ocurring. A similar but more general notion than what a measure gives us, is a random variable: a measurable function between two measurable spaces. I say "more general notion", since using the identity measurable function yields the measure we started with. A measurable function denoted $f : (\Omega_1, \Sigma_\Omega) \xrightarrow{m} (\Omega_2, \mathcal{G})$ $f : \Omega_1 \xrightarrow{m} \Omega_2$ is well defined when there is an ambient measurable space structure $(\Omega_1, \Sigma_\Omega)$ and $(\Omega_2, \mathcal{G})$. f allows us to talk about the distribution of a particular attribute of in the event space. For example $\Omega_1 = \{1, \ldots, 6\}^2$ may be the set of all events associated with playing 2 dice, while f may map to the sum of the two dice, $\Omega_2 = \{1, \ldots, 12\}$. A measurable function f, transforms any measure μ on $(\Omega_1, \Sigma_\Omega)$ to the pushforward measure μ' on $(\Omega_2, \mathcal{G})$. In order for this to always be well defined we require the following measurability to be satisfied by all measurable functions: $\forall G \in \mathcal{G}, f^{-1}(G) \in \Sigma_\Omega$.

A Lebesgue integral is a generalisation of the standard (Reimann) integral. Intuitively the lebesgue and reimann integral will coincide on the spaces such as $\mathbb{R}$, where the domain of integration is continuous. The lebesgue integral relaxes that requirement. The lesbesgue integral of a measurable function $f : \Omega \to [0, \infty]$ with respect to a measure $\mu : \Sigma_\Omega \to [0, \infty]$ is a number defined by:

$$\int_\Omega f \, d\mu := \sup \left\{ \sum_{i=1}^{n} f(\omega_i)\, \mu(U_i) \;\middle|\; \{U_i \in \Sigma_\Omega\}_{i=1}^{n} \text{ disjoint cover of } \Omega \right\}.$$

To get an intuition remember that integrals are supposed to add the values of function by breaking the domain up into infinitesimal pieces. Observe that the integral is a weighted sum of $f(\omega)$, weighted by the measure. The integral is iterating over the entire space by picking some choice of measurable disjoint sets whose union is the Ω (disjoint cover of Ω). And for each measurable set (U_i) it picks a representative point ω evaluates f on ω and weights it by how large the set is (μU_i). The supremum and infimum together enforce that the chosen disjoint cover is chosen as finely (small measure) as possible.

2.1.1 Borel spaces and lebesgue measures

2.1.2 Product of measurable spaces

2.2 Semantics of Probabilistic Programs

We first present APPL [Li et al., 2023, appendix A], a probabilistic programming language without bayesian updates. Syntactically it looks like a basic WHILE language. In Appl, there isn't an explicit `sample` construct, sampling is realised in the semantics of sequencing as explained later. Though the language is first order, PPLs still have a non-trivial, novel denotational semantics. The central difference is that types are interpreted as `MeasurableSpace` rather than just `Set` from which everything else follows. Concretely $[\![A]\!] = (\mathbb{A}, \Sigma_{\mathbb{A}})$. In our PPL we would like to express measures and manipulate measures. The type constructor GA is interpreted as the (meta) type $ProbMeasure[\![A]\!]$, the ~~`Set`~~ `MeasurableSpace` of measures over $[\![A]\!]$. So in order for GA to be a valid (object) type, there must be some canonical MeasurableSpace $\Sigma_{\mathrm{G}A} \subseteq \mathcal{P}(\Sigma_A \to [0, 1])$. This is indeed true, but we don't reprove the mathematical foundations on which the semantics stands, in this report.

`<insert: Why measurableSpaces?>`

$$
\begin{aligned}
A, B &::= A \times B \mid \mathit{bool} \mid \mathit{real} \mid A^n \mid \mathit{index} \mid \mathrm{G}A \\
p &\in [0,1] \\
r &\in \mathbb{R} \\
n &\in \mathbb{N} \\
\oplus &\in \mathrm{Arith} := \{+, -, \times, /, \hat{}\} \\
< &\in \mathrm{Cmp} := \{<, \leq, =\} \\
M, N, O &::= X \mid \mathrm{ret}\ M \mid X \leftarrow M;\ N \mid \\
&\quad (M, N) \mid \mathrm{fst}\ M \mid \mathrm{snd}\ M \mid \\
&\quad \mathrm{T} \mid \mathrm{F} \mid \mathrm{if}\ M\ \mathrm{then}\ N\ \mathrm{else}\ O \mid \mathrm{flip}\ p \mid \\
&\quad r \mid M \oplus N \mid M < N \mid \mathrm{unif}\ [0,1] \mid \\
&\quad [M, \ldots, M] \mid [M|N] \mid \mathrm{for}(n, M_{\mathrm{init}}, i\ X.\, M_{\mathrm{step}})
\end{aligned}
$$

Figure 2.1: Syntax of the language

$$
\begin{aligned}
[\![A \times B]\!] &= [\![A]\!] \otimes [\![B]\!] \\
[\![\mathbf{bool}]\!] &= (\{T, F\}, \mathcal{P}(\{T, F\})) \\
[\![\mathbf{real}]\!] &= (\mathbb{R}, \mathcal{B}(\mathbb{R})) \\
[\![A^n]\!] &= \underbrace{[\![A]\!] \otimes \cdots \otimes [\![A]\!]}_{n \text{ times}} \\
[\![\mathbf{index}]\!] &= (\mathbb{N}, \mathcal{P}(\mathbb{N})) \\
[\![\mathbf{GA}]\!] &= \mathcal{G}[\![A]\!] \\
[\![\mathbf{1}]\!] &= \text{the one-point space}
\end{aligned}
$$

Figure 2.2: Denotational semantics of types

$$
\begin{aligned}
[\![X]\!]\rho &= \rho(X) \\
[\![\text{ret } M]\!]\rho &= \text{ret } [\![M]\!]\rho \\
[\![X \leftarrow M;\ N]\!]\rho &= v \leftarrow [\![M]\!]\rho;\ [\![N]\!][X \mapsto v] \\
[\![(M, N)]\!]\rho &= ([\![M]\!]\rho,\ [\![N]\!]\rho) \\
[\![\text{fst } M]\!]\rho &= \pi_1([\![M]\!]\rho) \\
[\![\text{snd } M]\!]\rho &= \pi_2([\![M]\!]\rho) \\
[\![\text{T}]\!]\rho &= \text{T} \\
[\![\text{F}]\!]\rho &= \text{F} \\
[\![\text{if } M \text{ then } N \text{ else } O]\!]\rho &= \begin{cases} [\![N]\!]\rho, & [\![M]\!]\rho = \text{T} \\ [\![O]\!]\rho, & [\![M]\!]\rho = \text{F} \end{cases} \\
[\![\text{flip } p]\!]\rho &= \text{Ber}\, p \\
[\![r]\!]\rho &= r \\
[\![M \oplus N]\!]\rho &= ([\![M]\!]\rho) \oplus ([\![N]\!]\rho) \\
[\![\text{unif } [0,1]]\!]\rho &= \text{Unif}\,[0,1] \\
[\![[M_1, \ldots, M_n]]\!]\rho &= ([\![M_1]\!]\rho, \ldots, [\![M_n]\!]\rho) \\
[\![[M[N] : A]]\!]\rho &= \begin{cases} \pi_N\big([\![M]\!]\rho\big), & 1 \le [\![N]\!]\rho \le n \\ \text{arbitrary}(A), & \text{otherwise} \end{cases} \\
[\![\text{for}(n, M_i, i\ X.\, M_s)]\!]\rho &= \text{loop}(1,\ [\![M_i]\!]\rho,\ \lambda k.\ [\![M_s]\!]\rho[i \mapsto k,\ X \mapsto v]) \\
\text{where loop}(k, u, f) &= \begin{cases} \text{ret } u, & k > n \\ v' \leftarrow f(k, v);\ \text{loop}(k+1, v', f), & \text{otherwise} \end{cases}
\end{aligned}
$$

Figure 2.3: Denotational semantics of terms

2.2.1 Types in BPPL

The setting is the category of Measurable Spaces, `Meas`. All types in our language is interpreted as an object in this category. Syntactically:

$$\mathbb{A}, \mathbb{B} ::= \mathbb{R} \mid \mathcal{P}(\mathbb{A}) \mid 1 \mid \mathbb{A} \times \mathbb{B} \mid \cdots$$

Objects in this category are of the form (A, Σ_A). Morphisms are measurable functions. The interesting type here is $\mathcal{P}(\mathbb{A})$. It's the type of probability distributions over $\mathbb{A}$. Actually not quite, since we allow any measure $\mu : \Sigma_A \to [0, \infty]$, rather than $\Sigma_A \to [0, 1]$. Now in order to be an object in the category, $\mathcal{P}(\mathbb{A})$, the set of all measures on $\mathbb{A}$, must itself form a measurable space. There is a construction to form a σ-algebra for $\mathcal{P}(\mathbb{A})$ satisfying all the requirements (we don't go into it here).

A natural next question is how these measures can be created and manipulated. The semantics of PPLs are given in a monadic framework (in other words probabilistic programs can be thought of as effectful computations). In practice this means that there is are `ret` and `bind` type constructors to construct terms of (object) type $mathrmG$?, such that

$$[\![\text{ret}]\!] : A \to \mathcal{G}A \qquad [\![\text{bind}]\!] : \mathcal{G}A \to (A \to \mathcal{G}B) \to \mathcal{G}B$$

The core intuition of how probabilistic programs work is gained by studying the semantics of let bindings **??**. But first let's explain how variables work in Appl. A standard technique for representing variables is to use a variable context. A map from this context to the denotation of an AST would be considered a program. However, in Appl, the context is a measurable space, and the map is also measurable. Precisely the context and interpretation of well-typedness in context is given as:

$$[\![\Gamma, X : A]\!] = [\![\Gamma]\!] \otimes [\![A]\!] \qquad [\![M]\!] : [\![\Gamma]\!] \xrightarrow{\text{m}} [\![A]\!]$$

2.2.2 The let binding problem

Staton provided a compositional semantics for first-order Bayesian probabilistic programs in Staton [2017], which forms the foundation of program logics, validation of inference algorithms, etc. The scope of this project is entirely within first-order probabilistic programs (probability distributions over function spaces are not expressible). The programming language considered in Lilac uses a simpler semantics since it doesn't support bayesian updates. So in the following, we present the relevant excerpt of the semantics.

The key contribution of the paper is the introduction of `s-finite kernels`, motivated by validating a basic commutativity property that programmers expect. Specifically: when neither x

```
let x = t in          =     let y = u in
let y = u in                let x = t in
v                           v
```

Figure 2.4: commutative program transformation

is free in u nor y is free in t. Another benefit of this semantics is a term-wise compositionality, ie, the semantics of every subterm in a program is well-defined, not just whole programs. This is just another way of saying the semantics of open programs (those with free variables), as well as closed programs is well-defined, respectively. Validating proof rules in a logic is much better formulated in terms of such a compositional semantics. s-finite kernels denote open programs and s-finite measures denote closed programs.

At this point it is important to disseminate the difference between Lilac's lack of bayesian updates (call this APPL) and the more general language with bayesian updates (call this BPPL). As justified in the **??**, it is necessary to work with (unnormalised) measures when

2.2.3 Semantics of Let binding

?? Probabilistic programs are interpreted using the Giry monad Giry [1982] in the category of Measurable Space, `Meas`. Probability is an effect. This interpretation gives us the semantics of sequencing (let-binding) so we go through it below. Kernels also naturally show up in type of the monadic bind, which motivates it.

The Giry functor takes some object (measurable space) (A, Σ_A) to a measurable space on the measures of $(\Sigma_A \to [0, \infty], \cdots)$ (the σ-algebra is ommitted). We'll denote $\Sigma_A \to [0, \infty]$ as `Measure A`, for readability.

The explanation focuses on intuition through diagrams. For a measurabe space (A, Σ_A), a probability measure is of type $\mu : \Sigma_A \to [0, 1]$. This gives a distribution on A. The intuition behind μ is that we can sample some event using it. A kernel from A to B has type $\kappa : A \to MeasureB$. Each triangle below is a kernel. And the types of it input and output are indicated, and we compose the kernels along the type B as $\kappa \underset{B}{\circ} \iota$. The types of the induvidual kernels and composed kernel of Fig **??**, is given below

$$\iota : A \to \text{Measure } B \tag{2.1}$$
$$\kappa : B \to \text{Measure } C \tag{2.2}$$
$$\kappa \underset{B}{\circ} \iota : A \to \text{Measure } C \tag{2.3}$$

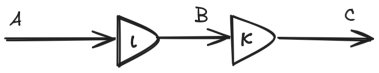

Figure 2.5: A simple probabilistic program

The interpretation of the kernel ι is that for every (deterministic) value of A it gets, it will give you a (probabilistic value) distribution over type B. So composition here, is sampling from the `Measure` B, getting a deterministic value of type B and passing that to the kernel κ. This sampling is done ranging over all the values of B in proportion to their distribution, and taking a weighted average of the values of type `Measure` C. More precisely, this is the integral: $\kappa \underset{B}{\circ} \iota = \lambda a. \int \lambda b.\ \kappa(b)\ d\iota(a)$ (integral will be explained in 2.1 in the final draft). Notice that $\circ$ has a type very similar to the monadic bind (in the Giry monad). We now write $\mathcal{G}(A)$ instead of `Measure` A, for the Giry functor. $\circ : ((B) \to \mathcal{G}(C)) \to ((A) \to \mathcal{G}(B)) \to \mathcal{G}(C)$. If we just swap the arguments and partially apply an $a : A$ on ι, we see that the bind of the Giry monad composes a measure with a kernel like so $\iota(a) \gg\!= \kappa : \mathcal{G}(B) \to (B \to \mathcal{G}(C)) \to \mathcal{G}(C)$

The most interesting part of Staton's commutative semantics is that when kernels have multiple inputs (this does extend into the realm of multi-categories but we don't go into that here). To clarify, these kernels have multiple deterministic values they take in order to return a single probabilistic value (a measure). The commutativity property illustrated by the equality of the two programs in Fig **??**.

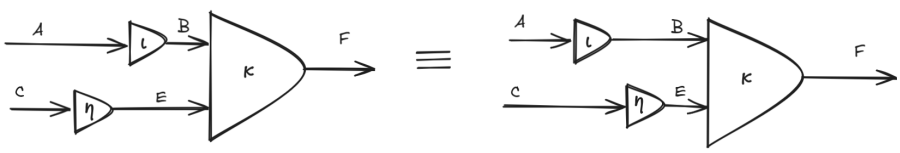

Figure 2.6: Commutativity of let bindings

$\kappa \underset{B}{\circ} \iota$ is given by the following on the left and right respectively

$\lambda a\ c. \int \lambda e.\ \left(\int \lambda b.\ \kappa(b, e)\ d\iota(a)\right) d\eta(c)$

$\lambda a\ c. \int \lambda b.\ \left(\int \lambda e.\ \kappa(b, e)\ d\eta(c)\right) d\iota(a)$

Though these look rather unreadable, all that's happening is swapping the order of integration, and there is a theorem which allows us to this in certain conditions. In Staton [2017] it is shown that this theorem holds if instead of general kernels, we restrict ourselves to s-finite-kernels. A kernel $\kappa : A \to \mathcal{G}(B)$ is finite if there is finite $r \in [0, \infty)$ such that, for all a, $\kappa(a, B) < r$. κ is s-finite if there is a sequence $\kappa_1 \ \ldots \ \kappa_n \ \ldots$ of finite kernels and $\Sigma_{i=1\ldots\infty}\ \kappa_i = \kappa$.

In short topological transformations of these diagrams corresponding to the motivating example in 2.4 do not change the semantics.

2.3 Separation Logic

Separation logic has been extremely productive in the last two decades in reasoning about imperative programming languages, after first being introduced in O'Hearn et al. [2001]. The classical separation logic introduced a compositional state, specifically a heap (modelled as a function $\mathbb{N} \to Val$), which was required along with a more usual variable environment, to interpret any assertion. This heap or more generally resource, had an algebraic structure to it: the most fundamental one being a partial commutative monoid (PCM), with the binary operator dictating when it is valid to combine to heaps. Many more complex algebras for resources were studied in later works Calcagno et al. [2007]; Pym et al. [2004]. Separation logic was identified as a natural tool toreason about imperative programs mutating heap based memory at a suitable level of abstraction. It then found success also in reasoning about concurrent programs Brookes and O'Hearn [2016].

2.3.1 Classical separation logic

The first separation logic was proposed by O'Hearn et al. [2001] for reasoning about programs which manipulate a heap. Before we move to probabilistic separation logic which is the main topic of the report, we first present separation logic in a simpler setting. Intuitively, separation logic is just first order logic with the additional connectives of the separating conjunction and separating implication (notation: $*$ and $-\!\!*$). These separating connectives are defined with respect to a *resource*. In the original separation logic we have a simple resource which is a finite partial function from some a set denoted Locations to a set Values (fig **??**). The notation $h_0 \star h_1$ is a partial binary operation denoting the union of h_0 and h_1 if they are disjoint. So we see that in order for a separating conjunct to hold we need to split the resource appropriately such that each part satisfies a conjunct. This is the principle behind all separation logics. Each proposition "owns" unique resources. One can imagine how each proposition makes assertions about parts of a program, and it is made explicit which parts of the heap, that part of the program "owns" and is thus able to modify. In short the logic allows reasoning in a resource-compositional manner. And specifically classical separation logic reasons about "memory" in languages like C, where the programmer must consciously manage the memory.

The other two *spatial connectives* are emp and $-\!\!*$. emp denotes the heap is an empty mapping, acting as the identity element to a monoid formed with $*$. $-\!\!*$ is to $*$ what $\to$ is to $\wedge$. Intuitively, $P -\!\!* Q$ holds under a heap q_- if P holds under some heap p for which the combined resource $q = p \star q_-$, satisfies the goal Q.

Next observe the definitions of *forall quantification*. We quantify over the set `Values` that the given variable used to define P can take. Though we haven't introduced the programming language that this logic reasons about, note that the variables form the bridge between the language and logic, since these variables are shared between programs and assertions we make about the program. We do not go into the specifics of the introducing the programming language for this particular logic, since that takes us too far afield.

2.3.2 Zoo of separation logics

In this section we see increasingly abstract presentations of separation logic to spot patterns amongst them, and give definitions of characterstics that help classify separation logics. Identifying these characteristics and "telling" Iris about it, lets us effectively use the proof automa-

$$\text{Heaps} \triangleq \text{Locations} \rightharpoonup_{\text{fin}} \text{Values} \qquad \text{Stores} \triangleq \text{Variables} \rightharpoonup_{\text{fin}} \text{Values}$$

(a) Basic sets and structures

$$\begin{array}{rcll} B & ::= & True \mid False & \\ E, F & ::= & \mathbb{N} & \\ P, Q, R & ::= & B \mid E \mapsto F & \text{Atomic Formulae} \\ & \mid & \text{false} \mid P \Rightarrow Q \mid \forall x.P & \text{Classical Logic} \\ & \mid & \text{emp} \mid P * Q \mid P \mathrel{-\!\!*} Q & \text{Spatial Connectives} \end{array}$$

(b) Syntax of separation logic

$$\begin{array}{lcll} s, h & \models & B & \text{iff } B \\ s, h & \models & E \mapsto F & \text{iff } [\![E]\!]_s \in \text{dom}(h) \text{ and } h([\![E]\!]_s) = [\![F]\!]_s \\ s, h & \models & \text{false} & \text{never} \\ s, h & \models & P \Rightarrow Q & \text{iff if } s, h \models P \text{ then } s, h \models Q \\ s, h & \models & \forall x.P & \text{iff } \forall v \in \text{Values}.\ [s \mid x \mapsto v], h \models P \\ s, h & \models & \text{emp} & \text{iff } h = [\,] \text{ is the empty heap} \\ s, h & \models & P * Q & \text{iff } \exists h_0, h_1.\ h_0 \star h_1 = h,\ s, h_0 \models P \text{ and } s, h_1 \models Q \\ s, h & \models & P \mathrel{-\!\!*} Q & \text{iff } \forall h'.\ \text{if } h' \star h \text{ exists and } s, h' \models P \text{ then } s, h \star h' \models Q \end{array}$$

(c) Semantic interpretation

Figure 2.7: Definitions of separation logic

tion/engineering developed by the Iris framework for our probabilistic separation logic (which will be presented later).

Abstract resources

It was hinted in the previous section emp and $*$ is like a monoid. In general, what we mean by a "resource" can be modelled with some algebraic structure. The spatial connectives $*$ and $-\!\!*$ are both defined using the binary operation $\star$. At the core of a separation logic is the partial commutative monoid (PCM) formed by emp and $\star$. A partial monoid is just a monoid accounting to the fact that $\star$ is a partial operation. So we guard all the laws with existence checks. This turns out to be quite messy in a formalism, as seen for example in the `assoc` law. The below code shows the formalisation used taken from Bppl.Lilac.Krm.

```
notation a:arg " ⋆ " b:arg => PcmBase.binop a b

class Pcm (α : Type*) extends One α where
  binop : α → α → Option α -- partial binary operation denoted `⋆`
  one_mul : ∀ a : α, 1 ⋆ a = a
  comm : ∀ a b, a ⋆ b = b ⋆ a
  assoc : ∀ a b c : α,
    ((⋆) <$> (a ⋆ b) <*> (some c)).join = ((⋆) <$> (some a) <*> (b ⋆ c)).join

class Krm (α : Type*) extends Pcm α, Preorder α where
  le_mul_mono : ∀ x x' y y' p' : α,
    x ≤ x' → y ≤ y' →
    (x' ⋆ y' = some p') → ∃ p, (x ⋆ y = some p) ∧ p ≤ p'
```

Lean follows a haskell like syntax. Above we define the typeclass representing PCM and KRM. `Pcm` extends `One`, which gives a $(1 : \alpha)$ which we interpret as `emp`. `assoc` is using functor syntax (from haskell) and `join : Option (Option α) → Option α`. This code is included to define precisely what is meant by partial monoid and krm, and to familiarise the reader with lean syntax.

The more interesting development is the order structure in the algebra. This can be motivated by wanting to model an affine separation logic (defined in the next section). Intuitively an affine separation logic always allows a larger heap to validate an assertion if a smaller heap does so too. This was not the case for the classical separation logic presented in the previous section which is a linear separation logic. Concretely only the single celled heap $\{1 \mapsto 42\}$ would validate the assertion $1 \mapsto 42$, and importantly $\{1 \mapsto 42, 2 \mapsto 42\}$ would not validate the same assertion. However, in an affine separation logic, any heap which contains $\{1 \mapsto 42\}$ would validate the assertion. Each of these flavours of separation logic are useful in different contexts. Classical separation logic is useful in a language with manual memory management while affine separation logic is used in a language with garbage collection. We see that we want an ordering in our algebra to express this "contains" relation or more generally how "informative" a resource is. So we introduce a preorder to the algebra, turning the PCM into a Kripke resource monoid (KRM) [Galmiche et al., 2005]. Note that KRMs also encompass PCMs/linear case, if we simply set the preorder to equality. For the affine heap-based separation logic, the preorder is subset inclusion on the domain of heaps. The standard connectives of separation logic (those connectives shared by all separation logics) can be generated from a KRM, which is the approach taken in the Lilac formalisation.

Logic of Bunched Implications

Abstracting even further, we would like to get rid of explicit mentions of the resource and the satisfiability relation $(s, h \models P)$ and talk instead about entailments of the form $\vdash P$ or $P \vdash Q$. The *logic of bunched implications* (BI) as presented by O'Hearn and Pym [1999] was what classical separation logic was based on. It can be described as a logic in which the familiar "intuitionistic" connectives of first order logic are combined with emp, $*$ and $-\!\!*$ (called spatial connectives, alluding to heaps), in the same logic. We can recover the notion of resources by our interpretation of logical entailment: $P \vdash Q$ is defined as (using connectives in the meta-logic) $\forall s, h (s, h \models P \Rightarrow s, h \models Q)$. The definition of $\vdash$ is in general specific to each logic, but in practice (for separation logic) usually has the above shape. Having established this greater level of abstraction, we can more succinctly define some characteristics of different separation logics which will come in handy when instantiating the Iris framework. All subsequent figures in this section are reproduced from [Krebbers et al., 2018].

First let's precisely define the relation between $*$ and $-\!\!*$ (identical to $\wedge$ and $\to$)

The rest of this section reproduces some figures from [Krebbers et al., 2018].

$$\text{emp} * P \dashv\vdash P$$

$$P * Q \vdash Q * P$$

$$(P * Q) * R \vdash P * (Q * R)$$

*-Mono

$$\frac{P_1 \vdash Q_1 \qquad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2}$$

*-Intro

$$\frac{P * Q \vdash R}{P \vdash Q -\!\!* R}$$

*-Elim

$$\frac{P \vdash Q -\!\!* R}{P * Q \vdash R}$$

<insert: If I add the one_le, does the KRM become an affine logic generator?>

Next we define the affine propositions: P as any proposition satisfying for all Q $P * Q \vdash Q$. In other words P can always be dropped. An affine separation logic is one where all propositions are affine. Affine propositions can also be equivalently defined as $P * \text{True} \dashv\vdash P$ or $\text{True} \dashv\vdash \text{emp}$. Another class of propositions is persistent propositions. Those propositions which may be duplicated to establish the goal. So for instance if P is persistent than $P \vdash P * P$. <insert: Why is this not the definition?>. Intuitionistic propositions are a final class of propositions which are both affine and persistent. Working with intuitionistic propositions in the premise feels like working with the familiar first order (intuitionistic) logic, since we don't at all need to keep track of how many times these propositions are used. Because of this Iris treats intuitionistic propositions specially, keep them separate from the rest of the "spatial" propositions in the premise. So a user of the iris proof-mode will see an "intuitionistic context", following rules familiar to them from using the host theorem prover (Rocq or Lean), and additionally the spatial context, which they will need to treat more carefully. Examples follow in the next section.

2.3.3 Iris and MoSeL

Iris is a general framework for tactic based proofs in a separation logic. Iris (specifically MoSeL) can be understood according to the layers of abstraction it provides.

```
KRM -> BI                                  ->                    Iris Proof Mode
    |_ Probability Specific connectives -> typeclass instances -|
```

This is sort of the general structure of how Iris is meant to be used by someone instantiating it. You start with a KRM or similar algebraic structure which is the resource model, from which there is pre-determined construction of a BI (those connectives common to all separation logic). Given this Iris proof mode will be aware of the standard properties of a separation logic. For example, it is able to prove the following goal, note it has implicitly applied both commutativity separating conjunct <insert: example 1> We are also able to introduce both variables and premises with the same command, to build a context of everything we have to prove our goal. <insert: example 2> What is unique about the Iris proof mode when compared to Lean or Rocq, is that it has an additional spatial context. <insert: example with spatial context> In short it is engineered to provide an interface which mimics the natural reasoning style of user. The IPM understands separation logic and endeavours to automate common boring tasks.

Though these seem like rather basic conveniences they are quite difficult to provide in a manner extensible to all separation logics. In order to understand how the proof mode works (as we will be instantiating Iris), it is first necessary to understand tactic based proofs in general.

2.4 Theorem Proving

2.4.1 Dependent type theory

Examples in this section are based on the excellent introduction to the calculus of inductive constructions by Paulin-Mohring [2014].

Foundational theorem proving is proving theorems with only the axioms (foundations) of mathematics assumed. Likewise foundational software verification for proving theorems (properties) of programs.

Here we focus on the theory of foundational theorem proving and in the next (2.4.2) we focus on how these ideas scale to large projects in practice. Note that we are interested in "proving theorems" as software verification is just a particular instance of theorem proving (using program logics, which is covered in detail in the rest of this report).

We now focus on a specific "Foundation of Mathematics" which has proved ideal for theorem proving in practice, as displayed by the most successful theorem provers - Rocq, Lean, Agda, Idris - all being based on dependent type theory. Dependent type theory has the remarkable property of encoding both propositions (synonymous to theorems) and proofs on one side and types and programs on the other, as part of the same language. These two sides are in-fact formally indistinguishable. The Curry-Howard isomorphism (or propositions as types method) is an observation that the rules governing logic (introduction elimination rules), are much the same as the inference rules governing typing judgements. So the two can be used interchangeably. Certain types in our type theory will be interpreted as propositions, and terms of that type will be proofs of that proposition!

Below is a description of Calculus of Constructions (CoC), a flavour of dependent type theory on which Rocq (CoC and its successors were developed for Rocq) and Lean are based. Agda is based on Martin Lof Type Theory (MLTT) which differs in a matter of predicativity. We will explain and revisit this briefly at the end.

Pure CoC consists of very few rules and constructs and it is very surprising that it is a valid foundation for all mathematics. We start with a typed lambda calculus in which both the terms and the types are given by the *same* syntax. This is known as a pure type system (PTS). Instead of function/arrow type we have the dependent product, $\Pi(x : A), B(x)$, where B may be defined using x. Additionally, (as there is no separate syntax for types) all types have *sort*. In other words, every construction in the calculus must have a type, which itself will have a type and so on, infinitely. A has type $\mathbf{Type_1}$ (or *sort* $\mathbf{Type_1}$, to acknowledge that A is itself a type). There is actually an infinite hierarchy of these types: $A : \mathbf{Type_1} : \mathbf{Type_2} : \mathbf{Type_3} \ldots$, where the subscripts are indices known as *universes*.

The type of identity functions A is given by $\Pi(\_ : A), A$. We use the notation $A \to A$ in this case since it is not dependent. For completeness the identity function is typed $(\lambda x : A \mapsto x) : \Pi(\_ : A), A$ As a first example of a dependent type take the type of the polymorphic identity function, which takes a type and gives the identity function for that type: $\Pi(A : \mathbf{Type_i}), A \to A$. So $A \to A$ (or $\Pi(\_ : A), A$) is using A in its definition therefore dependent. Again for completeness we type the term which is the polymorphic identity function: $\lambda A : \mathbf{Type} \mapsto \lambda x : A \mapsto x : \Pi(A : \mathbf{Type_i}), A \to A$. We will not go into the detail of universe polymorphism which strictly speaking should be considered since the index i is unquantifed in the above expressions.

We now move to expressing logic in CoC, but before doing so we introduce the sort $\mathbf{Prop} : \mathbf{Type_1}$, so that the (infinite) set of sorts is given by $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$. **Prop** is special and is *impredicative* as we will see in 2.4.1. We map first order logic propositions to types of sort **Prop**. For example, if $A : \mathbf{Prop}$ and $B : \mathbf{Prop}$, then logical implication $A \Rightarrow B$ corresponds to the type $A \to B$. This is justified by the meaning of implication to be being able to prove B given a proof of A. Forall quantification (also) corresponds to dependent product types: given some $T : \mathbf{Type_i}$ and $P : \mathbf{Prop}$, the logical statement $\forall x : T, P(x)$ corresponds to the type $\Pi(x : T), P(x)$. This also make sense since to prove a universally quantified statement we need a proof of $P(x)$ for every x, which is what a term of type $\Pi(x : T), P(x)$ would give. We will use the notation $\forall x : T, P$ for $\Pi(x : T), P(x)$, freely, as is standard in Lean.

Dependent type theory as a foundation is an example of *constructive mathematics* because proofs are always built by assembling the required data as seen in above example. With that intuition in mind lets show how some other logical connectives are represented:

As seen in this example, the standard recipe in constructive mathematics is to specify a proposition by what is required prove it. In the next example we see representation also arises from what the connective enables us to prove. Take False $:= \forall(P : Prop).P : Prop$. First notice that if we have a proof of False, that is a term of type $\forall(P : Prop).P$, we would get the proof of every any proposition P, out of that, encoding false entails anything. Also, it is not possible to construct a term of type $\forall(P : Prop).P$, ie, there is no proof of False. A strange thing that one might notice is that we are quantifying over all everything in $Prop$, in order to obtain something of type $Prop$.

Predicativity

In this section we continue with more examples of logic using dependent type theory since it comes up in a different context in the definition of Iris, which took a while to figure out.

Let's now introduce all the typing rules. Remember $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$.

$$\frac{\Gamma \vdash}{\Gamma \vdash \mathrm{Prop} : \mathrm{Type}_1}\ \textsc{Type-Prop} \qquad \frac{\Gamma \vdash}{\Gamma \vdash \mathrm{Type}_i : \mathrm{Type}_{i+1}}\ \textsc{Type-Form}$$

$$\frac{\Gamma \vdash x : A \in \Gamma}{\Gamma \vdash x : A}\ \textsc{Var} \qquad \frac{\Gamma \vdash A : s \qquad x \notin \Gamma \qquad s \in S}{\Gamma, x : A \vdash}\ \textsc{Ctx-Ext}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \mathrm{fun}\, x : A \Rightarrow t : \Pi x : A, B}\ \textsc{Fun-Intro} \qquad \frac{\Gamma, x : A \vdash B : \mathrm{Prop}}{\Gamma \vdash \Pi x : A, B : \mathrm{Prop}}\ \textsc{Pi-Prop}$$

$$\frac{\Gamma \vdash A : \mathrm{Type}_i \qquad \Gamma, x : A \vdash B : \mathrm{Type}_j}{\Gamma \vdash \Pi x : A, B : \mathrm{Type}_{\max(i,j)}}\ \textsc{Pi-Type} \qquad \frac{\Gamma \vdash f : \Pi x : A, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\, a : B[a/x]}\ \textsc{App}$$

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : s \qquad A \preceq B}{\Gamma \vdash t : B}\ \textsc{Conv}$$

Figure 2.8: Inference rules for the purely functional part of CIC [Paulin-Mohring, 2014]

We notice that there are two typing rules for the dependent product: `PI-PROP` for sort Prop and `PI-TYPE` for all other sorts. `PI-TYPE` is saying that the dependent product falls into a sort which is the max of the sort of its domain and codomain. While there is also a special case rule `PI-PROP`, which says that `PI-PROP` says that regardless of the domain, if the codomain is in Prop, then the Π-type stays in Prop. We say that the sort Prop is impredicative, because elements in it are defined by quantifying over Prop itself! Remembering False := $\Pi P : Prop, P : Prop$, we see exactly this. And suppose that we instead were to type this with the rule `PI-TYPE` and $Prop \triangleq Type_0$, then the domain, $Prop : Type_1$ and codomain $P : Prop$ would give the Π-type sort $Type_1$, making Prop *predicative*. However, we would like $False : Prop$, so the impredicativity of Prop is desirable. Let's also recall the definition of forall quantification which also depended on predicativity: $\forall x : T, P$, we can quanitfy over values in much larger universes or quantify over all Props and still end up with a type in Prop. Impredicativity is powerful but fragile. It can be shown that if $Type_1$ is also impredicative, then this type theory would become inconsistent [Coquand, 1999].

As a final example lets have a look at $A \wedge B \triangleq (P : Prop).(A \to B \to P) \to P : Prop$. Or making things a little more explicit: $\wedge := (AB : Prop).\ (P : Prop).(A \to B \to P) \to P : Prop \to Prop \to Prop$, where we are first using lambda abstractions and then a Π-type construction.

Proof Relevance

Finally, Proof Relevance separates the type theories of Lean and Rocq. Lean is Proof irrelevant meaning terms of types of Prop, are definitionally equal. So for instance if we have a structure bundling data and a proof of a property like so in Lean:

```
structure MeasurableFun (α β : Type*) [MeasurableSpace α] [MeasurableSpace β] where
  /-- underlying function -/
  toFun : α → β
  meas: Measurable toFun
```

the equivalence of two `MeasurableFuns` is determined solely by the equality of the `toFun`, since `meas` is a (dependent) proof and will be definitionally equal if `toFun` is equal. In practice this is incredibly convenient, and is applied automatically throughout the codebase to simplify proof objectives. It was always used in an elegant abstraction in establishing that certain constructions form a KRM, which we will detail later.

The choices made in lean's type theory to bring about proof irrelevance does have some major theoretical drawbacks such as leading to undecidability of definitional equality [Carneiro, 2019, see chatper 3], which however don't seem to matter too much in practice.

Calculus of Inductive Constructions

Mention here that 1) Inductive families... 2) strict positivity

In this section we hope to have gotten across that theorem proving in a dependent type theory is exactly type checking of terms constructed by the user. This is all that is needed to motivate the next section on how this is partially automated and scaled in Rocq and Lean (going into more Lean specific detail). We also introduced the notions of predicativity and proof relevance, both of which I encountered over the course of the project without have previously heard of it. Interestingly they also cleanly separate 3 of the most significant proof assistants. They will be mentioned breifly where relevant but is not particularly important in understanding most of this report.

2.4.2 Metaprogramming

We now know that constructing a proof involves assembling many subproofs according to the typing rules to obtain a term of a desired Type (actually Prop). It is however extremely time-consuming and error-prone to go down to this level of detail when constructing a proof. Pen and paper proofs operate at a much higher level, and the use of metaprogramming in Proof assistants, is motivated by the desire to remove the mental overhead of mechanised proofs, move closer to the style of pen and paper proofs.

"Tactics" are metaprograms meant for constructing proof terms. A Tactic block is started in Lean using *by* syntax: in our code if we wanted to a term (proof) of type $A : Prop$, we can write $(by \ldots) : A$ (though the type A, would usually be inferred by lean). So how does the *by* tactic make it easier to construct the term? It provides a proof context, of premises and goals, which is actually additional state wrapped in a monad called `TacticM`. Inside a tactic block a sequence of `TacticM` monads are composed, each monad constructed using one Lean's ecosystem of tactics, incrementally modifying the proof state until "all goals are solved". And at the end of a tactic block, using the information acquired in the monadic state a term of the desired type should be constructed using the information collected in the TacticM monad.

We have been pretty vague about what this "proof context" and "goals" actually are so far. In lean there are a few more constructors for terms than in the simply types lambda calculus, on top of lambda abstraction and application. Terms are called `Expr` in lean. We introduce the constructor `mvar : MVarID \to Expr`. For building an expression given a metavariable. This is at the heart of how tactics work. Tactics communicate to eachother of what has been proven and what is left to prove using metavariables. We will illustrate this through a simple example reproduced from [leanprover community, 2024, chapter 4]. In the proof of the following theorem we show the evolution of the proof context, line by line.

```
example {α} (a : α) (f : α → α) (h : ∀ a, f a = a) : f (f a) = a := by
  apply Eq.trans
  · apply h
  · apply h
```

Metavariables are used to represent unknown terms of known types, which may be holes in larger terms or a goal we want to prove (term of a certain Prop that we are looking to construct). As soon as the tactic block starts with **by**, a metavariable `?m1` which is unassigned but has type `f (f a) = a`. Now we look at the state after the first `apply`. The types of `Eq.trans` and `?m1` can be unified, so the tactic succeeds and we have new proof state as follows:

To a user it would look like this:

```
...
⊢ f (f a) = ?b

...
⊢ ?b = a

...
⊢ α
```

But its helpful to see the state of metavariables explicitly:

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 : f (f a) = ?b
?m3 : ?b = a
```

```
?b
```

where `Eq.trans : ∀ (α : Sort u), ∀ (a b c : α), a = b → b = c → a = c`.

The old goal `?m1` has been assigned (albeit with holes in the assignment). Two new goals ?m2 and ?m3 were created and an additional mvar ?b, which is not a Prop was created as well. After the second `apply`, which acts on the goal ?m2, we assign ?m2 and also assign ?b through the unification process.

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 := h (f a)
?m3 : ?b = a
?b  := f a
```

Only ?m3 needs to be assigned which is taken care of the final `apply`. At the end of the tactic block, we obtain the term `Eq.trans α (f (f a)) (f a) a (h (f a)) (h a)` which is typed as `f (f a) = a`, as desired!

Chapter 3

Formalisation of Lilac

3.1 Syntax and semantics of Appl

To reason about Appl (see 2.2), we first need to define Appl in Lean. A first attempt at doing this may be inspired by how compilers are written: define an AST for the syntax of the language, and a typechecker for accepting or rejecting AST terms. Following such an approach would be painful given our goal of specifying properties of programs. We would always have to guard for the possibility that a program is not well-typed, in the assertion, and would lead to a proliferation of exception cases. Fortunately we are working in a dependently typed language, which is expressive enough to circumvent the problem by letting us define a syntax (AST) that is well-typed by construction.

For disambiguation, we refer to types in the *meta language* (in our case Lean) and *object language* (Appl), when it is unclear.

There are several standard ways to define object languages which are well-typed by construction, which all differ in the crucial problem of representing (object language) variables. This seemingly innocent problem of how to represent variables is one of the biggest issues encountered in this project, and standard techniques don't quite work for Appl, because of it's non-standard interpretation of (object language) types.

The final (Lean) representation of chosen for Appl follows a technique called Dependent De Brujin Indices [prefix Chlipala, 2013, see chap 17, Reasoning about programming language syntax]. Discussion on the alternatives are postponed to **??**.

First we present the types of Appl along with its denotational semantics, taken from Bppl.Lilac.Appl.

```
inductive Ty where
  | prod : Ty → Ty → Ty
  | bool
  | real
  | exp : ℕ → Ty → Ty -- Ty^n
  | index
  | G : Ty → Ty

@[reducible] def Ty.den : Ty → MeasCat
  | prod ty₁ ty₂ => .of (ty₁.den × ty₂.den)
  | bool => .of Bool
  | real => .of ℝ
  | exp n ty => .of (∀ (_ : Fin n), ty.den) -- using MeasurableSpace.pi
  | index => .of ℕ
  -- using `MeasureTheory.ProbabilityMeasure.instMeasurableSpace`
  | G ty => .of (ProbabilityMeasure (ty.den))
```

This mirrors the definition in the paper with no deviations. Note that the denotation of Appl types is `MeasCat`, the category of MeasurableSpace, rather than **`Type`** as for most other object languages. It is crucial to annotate `@[reducible]`, which tells the typechecker to unfold `Ty.den` when checking if two types are definitionally equal. The importance of this will be made clear once we describe the denotation of terms.

First we describe the De Brujin indices

Here is an excerpt of the definitions of terms, (omitted constructors can be found on Github).

```
inductive Term : List Ty → Ty → Type
  | var : Member ty ctx → Term ctx ty
  | ret : Term ctx ty → Term ctx (.G ty)
  | bind : Term ctx (.G ty₁) → Term (ty₁ :: ctx) (.G ty₂) → Term ctx (.G ty₂)
  | pair : Term ctx ty₁ → Term ctx ty₂ → Term ctx (.prod ty₁ ty₂)
  | fst : Term ctx (.prod ty₁ ty₂) → Term ctx ty₁
  | snd : Term ctx (.prod ty₁ ty₂) → Term ctx ty₂
  | T : Term ctx .bool
  | F : Term ctx .bool
  | ite : Term ctx .bool → Term ctx ty → Term ctx ty → Term ctx ty
  | flip : (p : R≥0) → (h : p ≤ 1) → Term ctx (.G .bool)
  | r : ℝ → Term ctx .real
  | arith : Arith → Term ctx .real → Term ctx .real → Term ctx .real
  | cmp : Cmp → Term ctx .real → Term ctx .real → Term ctx .bool
  | unif01 : Term ctx (.G .real)

/-- Takes the term and variable environment (which is a measurable function)
to give an element of a measurable space -/
@[simp] def Term.den : Term ctx ty → List.TProd (⟪·⟫) ctx -m→ ty.den
  | var mem => ⟨fun env ↦ env.get mem, List.TProd.measurable_get mem⟩
  | ret X => ⟨fun env ↦ probDirac (X.den env), measurable_probDirac.comp X.den.2⟩
      -- MeasureTheory.Measure.measurable_dirac.comp X.den.2
  -- In `bind` we are working exactly with kernels (as shown by the two coercions above)
  -- The `Kernel` api bundles `measurable` property and we need to convert to it and back
  -- to use its deep measure theoretic results for constructing measurable functions
  | @bind _ ty₁ ty₂ M N =>
    letI T := List.TProd (⟪·⟫) ctx
    letI k₁ : Kernel T (⟪ty₁⟫ × T) := (toMK M.den ×ₖ Kernel.id)
    letI k₂ : Kernel (⟪ty₁⟫ × T) ⟪ty₂⟫ := toMK N.den -- need to provide type annotation for this
    toDen (k₂ ∘ₖ k₁)
  | pair M N => ⟨fun env ↦ (M.den env, N.den env), Measurable.prod M.den.2 N.den.2⟩
  | fst M => ⟨fun env ↦ (M.den env).fst, Measurable.fst M.den.2⟩
  | snd M => ⟨fun env ↦ (M.den env).snd, Measurable.snd M.den.2⟩
  | T => ⟨fun env ↦ true, measurable_const⟩
  | F => ⟨fun env ↦ false, measurable_const⟩
  | ite P M N => ⟨fun env ↦ if P.den env then M.den env else N.den env,
      -- the function translating from bool to Prop is `Measurable`
      -- by `measurableSet_singleton true`
      Measurable.ite (P.den.2 (measurableSet_singleton true)) M.den.2 N.den.2⟩
  | flip p hp => ⟨fun _ ↦ ⟨(bernoulli p hp).toMeasure, inferInstance⟩, measurable_const⟩
  | r x => ⟨fun _ ↦ x, measurable_const⟩
  | arith op M N => ⟨fun env ↦ op.den (M.den env, N.den env),
    (op.den.2).comp (M.den.2.prod N.den.2)⟩
  | cmp op M N => ⟨fun env ↦ op.den (M.den env, N.den env),
    (op.den.2).comp (M.den.2.prod N.den.2)⟩
  | unif01 => ⟨fun _ ↦ ⟨lebI, inferInstance⟩, measurable_const⟩
```

interesting because of the `theorem foo : True := by trivial` Some things that are noteworthy are the fact

3.2 Semantics of Lilac propositions

3.2.1 Reviewing Probabilistic Separation Logics

More recently there has been a lot of research into probabilistic separation logics: [Barthe et al., 2019], introduced separation as probabilistic independence. [Li et al., 2023] introduced Lilac, which allowed usage of continuous distributions, providing a measure theoretic definition of resources. [Ho et al., 2025] introduced reasoning about bayesian updates (supporting the `observe` and `normalize` constructs), thereby providing an axiomatic semantics for the general form of first order probabilistic programs (ie, BPPL as introduced by [Staton, 2017]).

There is a parallel line of work in relational separation logics. One could argue that relational logics are more important in the probabilistic domain, since in bayesian probabilistic programing (so including `observe` and `normalize`), frequently generates probability distributions which have no closed form solution! Sampling from these can only be done using approximation methods like Variational Inference. The justification for relational logics classical is that relational properties are expressed more naturally, but could perhaps still be done using unary reasoning, in an inelegant

way. In probabilistic programs however, relational reasoning might in fact be more expressive making it important. Relational reasoning for probabilistic programs (though not using a separation logic) can be found here Barthe et al. [2009]. More recently there has been a novel combination of relational and unary reasoning for probabilistic programs in a separation logic by [Bao et al., 2025].

3.2.2 Lilac

The following are a few core intuitions behind Lilac. Lilac aims to reason about random variables, which is a higher level of abstraction that we generally reason in informally (as opposed to descending down into measure theory and lebesgue integrals). Since it's a separation logic, it is also compositional but along a different axis to compositionality of the denotational semantics. I call the compositionality of probabilistic separation logic a line-by-line compositionality (as opposed to term-wise compositionality). Each let binding creates a new "line" and we can compose our reasoning about each of line. In the denotational semantics we would instead reason compositionally about the arguments to the monadic bind operator (an s-finite measure and an s-finite-kernel). The former is a more natural way to reason about a sequencing operator.

This reasoning style is achieved by defining a resource which gets carried through and composed and split according to the laws of separation logic. This resource or state is a canonical source of randomness, the Hilbert cube, along with a measure space structure over it. It can be argued that the Hilbert cube could be replaced with something else, and indeed the definitions are parametric over this choice of measure space for a while. Any distribution can be constructed from the Hilbert cube (which is like a uniform distribution) using the operations/terms of the language.

`<insert: ownership as measurability>`

`<insert: independent combinations of probability spaces>`

3.3 Instantiating Iris

Iris Jung et al. [2018] is a step-indexed modal separation logic partly designed as a framework for embedding other separation logics into, to take advantage of the Iris Proof Mode (IPM) Krebbers et al. [2017]. The Iris proof mode provides state of the art "tactical support", tactic based interactive theorem proving. More recently the front of the IPM was abstracted from Iris to any separation logic to create MoSeL Krebbers et al. [2018]. We aim to use MoSeL in this project for several reasons

1. Neither Lilac nor BaSL is step-indexed, so it is much cleaner to instantiate MoSeL rather than Iris
2. Making Lilac compatible with Lilac's interface (instantiating an algebraic structure called CMRA Jung et al. [2018]) is an unsolved problem (future work section Li et al. [2023])
3. MoSeL was completely ported to Lean, shortly before the start of this project! König [2022].

MoSeL presents the interface of a "logic of Bunched Implications" which essentially contains all standard connectives of a first order logic and the separation logic connectives. The only addition MoSeL adds to this is a persistence modality (which can be instantiated in a trivial way in exchange for defeating any automation it might achieve?).

One of the key generalisations that MoSeL makes over Iris is parametricity over linear and affine logics. Iris is an affine logic meaning separating conjuncts in a premise can be used 0 or 1 time in proving a goal; they can be thrown away if desired. In a linear logic however every conjunct must be used exactly once, as in the first separation logic by O'Hearn et al. [2001]. There is naturally quite an emphasis on describing whether a logic is affine or linear, since throwing away unwanted predicates when it is suitable to do so, is exactly the kind of thing a user wants automation for. In BaSL Ho et al. [2025], there is a short discussion stating BaSL is part affine and part linear. Precisely characterising the constructs which are affine and linear or under what conditions should be explored for better tactical support.

MoSeL like IPM before it provides tactics mirroring the native Lean tactics, such as `iIntros` and `IAssumption`

In an ongoing project to port Iris from Rocq to Lean, MoSeL was completely ported to Lean right before the beginning of this project leanprover-community [2026]; König [2022]. So the formalisation builds on this. The interface provided by MoSeL is quite general. The interface defines a pre-orderered Set (PROP, Entails)

```
class BIBase (PROP : Type u) where
%   ⊢ : PROP → PROP → Prop -- Entails
  pure : Prop → PROP
  ∧ : PROP → PROP → PROP
  → : PROP → PROP → PROP
  * : PROP → PROP → PROP -- separating conjunct
  -* : PROP → PROP → PROP -- magic wand
  persistently : PROP → PROP
  ...
```

The type class `BIBase` defines the interface for the "data" parts of the separation logic, while the axioms that these need to satisfy (the "proof" parts) are defined in an extension of this type class. We are defining the separation logic, Lilac or BaSL, in the meta-logic, Lean. So `BIBase` is parametrised by a type `PROP` which is a proposition in the separation logic (as opposed to `Prop` in lean).

We draw the reader's attention to the `Entails` which allows us to provide the semantics of entailment. The soundness of MoSeL's syntactic entailments are established by the semantics provided when instantiating `Entails`. This gives us a little flexibility in how we'd like to instantiate `PROP`. For simpler logics we could have `PROP`, directly capture the semantis of that proposition, giving `PROP` this shape.

```
PROP := Store → Heap → Prop
```

My understanding is that this precisely encodes a relation: the satisfiability relation. An n-ary relation is encoded as a function from the n parameters to a Proposition. In other words it's a set of n-tuples. (Sets of type A are encoded as $A \to Prop$). Notice however that $s, h \vDash P$ should be of the form

```
PROP := Store → Heap → PROP → Prop
```

where did the P (of type `PROP`) go? It's given implicitly through the inductive construction of a term of `PROP` through the constructors defined by `and`, `sep`, etc. Any term of `PROP` implicitly contains the tree of constructors used to create it.

This is standard for the simple example instantiations of MoSeL (such as classical separation logic), but quite convoluted conceptuall. We may wish to split the representations of the separation logic assertions into syntax and semantics, which is what is done in non-trivial logics and this projects formalisation. There is a denotation function of type $Store \to Heap \to PROP \to Prop$ (exactly how the satisfiability relation should look) used in the instantiation of Entails; much more natural!

3.4 Pi-lambda-theorem as an induction principle

The π-λ-theorem is tool in measure theory which can be used for equality of two measures. We use it prove a certain uniqueness of measures theorem, used to establish that separating conjunction yeilds a PCM (partial commutative monoid) structure. It is similar to an induction principle on the structure of a σ-algebra. So we need prove some property C holds in the base and inductive cases of a σ-algebra. However there isn't an inductive type underlying this induction principle! The reason is two fold 1) when generating a measurable set from some base measurable sets, a specific set can be shown to be measurable by multiple different permutations of application of the axioms. For reference below is included an inductive defintion generating a σ-algebra from base measurable seats.

```
/-- The smallest σ-algebra containing a collection `s` of basic sets -/
inductive GenerateMeasurable (s : Set (Set α)) : Set α → Prop
  | protected basic : ∀ u ∈ s, GenerateMeasurable s u
  | protected empty : GenerateMeasurable s ∅
  | protected compl : ∀ t, GenerateMeasurable s t → GenerateMeasurable s tᶜ
  | protected iUnion : ∀ f : ℕ → Set α, (∀ n, GenerateMeasurable s (f n)) →
```

```
GenerateMeasurable s (⋃ i, f i)
```

2) The π-λ-theorem provides a relaxation on our proof obligation. We dont prove that the property C holds for countable unions (`iUnion` above) if it holds for each of the sets we're taking a union over. Instead we only need to do so for countably pairwise disjoint unions.

Being closed under disjoint unions instead of unions, is the difference between a λ-system compared to a σ-algebra. This relaxtion is the content of the π-λ-theorem which we don't go into here. However there is an alternative "inductive" definition of a λ system with the following axioms:

1. The system $\mathcal{C}$ containt the base measurable sets we chose
2. be closed under proper difference, for $\forall V, U \in \mathcal{C}, V \subseteq U \to U \setminus V \in \mathcal{C}$
3. closed under increasing limits, take $i \in \mathbb{N}$ and $U_i \in \mathcal{C}$ then $\bigcup_{i \in \mathbb{N}} U_i \in \mathcal{C}$

Unfortunately this is not the definition of λ-system in Mathlib, and it doesn't provide the "induction principle" according to this definition. For the case of probability measures (μ (universal set) = 1), there is a simple workaround, which is sufficient for Lilac. However BaSL uses unnormalised measures since it support bayesian conditioning, and so, there may be no simple workaround but reproving this alternative induction principle.

3.5 Binder Discipline

The choice between Dependent De Brujin Indices and Parametric Abstract Higher Order Syntax (PHOAS).

3.5.1 Dependent De Brujin Indices

- More straightforward to the uninitiated following a little more intuitively from the paper. Important if we want this is to be an accessible code base for future probabilistic separation logics to follow from.
- Measurability of maps expressible explicitly

3.5.2 PHOAS

- Notion of substitution inbuilt
- Example programs written in it are naturally readable. No separate parser and translation from a normal lambda calculus required. Correctness of the parser doesn't need to be proven... (but further consideration on this point requires deciding if there will be a separate executable implementation of the APPL syntax)

3.6 Instantiating Iris

Theorem proving, can be split into two activities: writing definitions and proving theorems. The former doesn't seem difficult or time-consuming. It sounds reasonably easy to come up with a skeleton for the codebase, based on a paper. After a while working on this project I have realised this is not true, and indeed this is well known by both the theorem proving community, also true for professional mathematics and software engineering. Let's elaborate on the latter. Writing computer code is parallel to writing definitions (of data and algorithms). So definitions are indeed important and time-consuming in Lean as well. There is no parallel to writing theorems though, programs just need to convince themselves of the correctness.

A milestone was reached when the definitions of the language and logic were written down, with a set of abstractions to convince myself this development was sound. These definitions were composed of a separate syntax and semantics for both the language and the logic, using dependent de brujin indices for both. These definitions would have sufficed to prove formally the correctness of all the proofs in the paper. However, of the project is not only to validate the soundness of the logic, but also to create a usable tool for proving properties of actual probabilistic programs ergonomically (using tactics). The best way to do so is to instantiate the Iris framework. And the interface

provided by Iris was not compatible with current definitions. Below we explain why it wasn't compatible and how Iris' interface is motivated.

3.6.1 Deep vs Shallow Embeddings

When defining a language or logic, within a meta-language (lean in our case), we are presented with 2 choices. A deep embedding; inductively define the syntax, and the denotational semantics becomes an inductive definition over the syntax. For clarity, here is an excerpt from a deep embedding of the Lilac logic:

<insert: complete this (code from c3ca63f)>

```
inductive Assertion : List Ty → List Ty → Type
  | top : Assertion ds rs -- ds: deterministic context, rs: random variable context
  | and  : Assertion ds rs  → Assertion ds rs → Assertion ds rs
  | sep  : Assertion ds rs  → Assertion ds rs → Assertion ds rs
  | forall (d : Ty) : Assertion (d :: ds) rs → Assertion ds rs
  | forall_rv (r : Ty) : Assertion ds (r :: rs) → Assertion ds rs
  | exists_rv (r : TyRand) : Assertion ds (r :: rs) → Assertion ds rs
  ...

def Assertion.denote {ds : List TyDet} {rs : List TyRand}
    (P : Assertion ds rs) (γ : HList (⟦·⟧) ds) (D : RV (List.TProd (⟦·⟧) rs)) (ψ : PSp HC) : Prop :=
  match P with
  | top  => True
  | and P Q => P.denote γ D ψ ∧ Q.denote γ D ψ
  | sep P Q => ∃ ψ₁ ψ₂ : PSp HC, ψ₁ * ψ₂ ≤ some ψ ∧ P.denote γ D ψ₁ ∧ Q.denote γ D ψ₂
  | forall d P => ∀ x : ⟦d⟧, P.denote (x :: γ) D ψ
  | forall_rv r P => ∀ X : RV ⟦r⟧, P.denote γ (X ; D) ψ
  | exists_rv r P => ∃ X : RV ⟦r⟧, P.denote γ (X ; D) ψ
```

Figure 3.1: Excerpt of Deep Embedding of Lilac

On the other hand a shallow embedding where we fuse inductive structure of the syntax and the semantic denotation function together to simply. The denotations of terms in our object language get expressed directly in the meta language, and we lose the ability to distinguish entities in the object language from the meta-language. The previous point, in concrete terms is that we loose the ability to do induction over the object language now. Here is an excerpt of Lilac as a shallow embedding:

The shallow embedding had a much smaller surface and generally allowed the logic to piggyback on the forall and exists quantifiers of the meta language.

3.6.2 BI in Iris

Remember that in 2.3.2 we explained that BI (logic of bunched implications) is just abstraction which defining the standard connectives shared by every separation logic. Here is the definition of BI as a typeclass in iris-lean (a dependency of this project).

```
/--
Require the definitions of the separation logic connectives and units on a carrier type `PROP`.
-/
class BIBase (PROP : Type u) where
  Entails : PROP → PROP → Prop
  emp : PROP
  pure : Prop → PROP
  and : PROP → PROP → PROP
  or : PROP → PROP → PROP
  imp : PROP → PROP → PROP          -- implies
  sForall : (PROP → Prop) → PROP
  sExists : (PROP → Prop) → PROP
  sep : PROP → PROP → PROP          -- separating conjunct
  wand : PROP → PROP → PROP         -- magic wand
  persistently : PROP → PROP
  later : PROP → PROP
```

Firstly `Prop` is the universe of propositions, part of Lean's syntax. While `PROP` denotes the proposition in the object logic (Lilac in our case). The typeclass is called `BIBase` since there

is another typeclass `BI` extending `BIBase` which states the axioms of BI. As explained in 2.3.2, the instantiation of `Entails` can interpret a BI as a separation logic. `Entails` says what it means for a `PROP` to be true by translating into the meta language's **`Prop`**. `persistently` and `later` is omitted since the former is an orthogonal concern and the latter is irrelevant. `emp` can be understood as the unit of the monoid with which we will instantiate BI. There are several binary operators. The most important connectives are `sForall` and `sExists`. The original defintion from iris-rocq is more straightforward:

```
iForall : ∀ A : Type u', (A → PROP) → PROP
iExists : ∀ A : Type u',  (A → PROP) → PROP
```

We quantify over the type A of variables that we want to quantify over, since iris can't know what type a variables a specific logic may want to quantify over, and indeed as in Lilac there may be multiple types (Lilac has two of each quantifier). We have named the above quantifier `iForall`, with `i` short for indexed, since when quantifying over types we refer to the types (like `A`) as *type indices*. `iForall` and `iExists` is impredicative (see 2.4.1), meaning we quantify in universes larger than that of the type being constructed. `u'` is universe level, a natural number. Indeed, impredicativity is required here to provide complete flexibility in what variables an instantiating logic can then quantify over.

Now lets look at `sForall` and `sExists`, with `s` short for *set*. Note that in theorem provers, Sets (say `Set A`) are represented as `A → Prop`. So the signature for the quantifiers says that given a set of `PROP` we construct a `PROP`. The reason for this less intuitive version of quantification is that iForall (iExists) can be defined in terms of sForall (sExists). And the quantified types are not restricted to a predetermined universe `u'`.

3.6.3 First Attempt: Deeply embedded logic instantiation

There is a result in Lilac [Li et al., 2023, see Appendix B.10] called the substitution lemma which uses induction over the syntax of Lilac propositions. Supposing that induction over the syntax of the logic is important/necessary, a next step is to try to instantiate `BIBase` above with the deep embedding from Fig 3.1. We may try something like so:

```
instance (ds rs: List Ty) : BIBase (Assertion ds rs) where
  Entails P Q    := ∀ γ D ψ, P.denote γ D ψ → Q.denote γ D ψ
  and            := Assertion.and
  ...
```

Where we simply instantiate each of the constructors with syntax, and called the denotation function in `Entails`. This works nicely until we get to the instantiation of the quantifiers. We now need some syntactic version of `sForall` and not just the **`forall`**/`forall_rv` from Fig 3.1. Though the latter is all we need for our logic instantiating iris demands the flexibility to quantify over any type, not just the object language types. Let's introduce the following syntax into our language:

```
inductive Assertion' : List TyDet → List TyRand → Type
  | sForall : (Assertion' ds rs → Prop) → Assertion' ds rs
  ...
```

This looks like a valid lean definition, but it is not, since it fails a syntactic requirement of inductive types called *strict positivity*. The type being defined inductively may only appear in *postiive* locations, which means they are not allowed to occur in the input of an arrow type. Even if we suppose the `iForall` definition was used in BI, we still end up with nuanced problems with mixing semantics into the syntax because of the quantifiers. We spare further details, but the conclusion is that the impredicativity of quantifiers in `BI` disallows usage of a deep embedding of the logic.

3.6.4 Shallow embedding of the logic

3.7 Deparametrisation of LProp

The development got quite far with the `LProp` being parametrised with the type of the deterministic context. However, that started causing serious issues when stating the proof rules, because of the book-keeping that needs to be done with always keeping track of the number of variables in

context. Not to mention this bookkeeping is quite difficult since we always need to keep proving that the the contexts are a measurableSpace.

3.8 Weakest precondition

In order to reason about programs, the logic must be able to express facts about the program effectively. In other words the language and the logic need to "communicate" somehow. We would like to be very explicit here about how that happens, since:

- this is generally skipped from a first presentation of program logic since its an implementation detail. However, in a formalisation it is crucial as it forms the interface between two significant pieces of the project (the language in Bppl.Lilac.Appl and the logic in Bppl.Lilac.Assertion)
- Lilac unlike any other separation logic I am aware of, does not have a one-to-one correspondence between its variables in the language and the logic. If in the language a variable has type A then in the logic it has type of the random variable of A (precise definitions will follow)

The link between the language and the logic is established using a connective in the logic, which takes the denotation of a program and some other assertions regarding it. Lilac is a functional language

Perhaps the more intuitive form of these connectives is the Hoare triple which takes 3 arguments, represented syntactically as $\{P\}[\![M]\!]\{Q\}$. P (precondition) and Q (postcondition) are both propositions and $[\![M]\!]$ is the denotation of some program M. But here we will present instead the weakest precondition (wp) connective, in terms of which Hoare triples can be defined. wp is more amenable to automation and since it only takes in a single proposition.

3.9 Deparametrisation of LProp

`<insert: Give examples of how the old LProp caused serious issues>`

3.10 Variables

An insightful aspect of Lilac is to consider how it treats variables. Unlike usual program logics, Lilac does not have a one-to-one correspondence between variables in the language and the logic. And this is a reflection of the perspectives taken probabilistic reasoning. One thinks about random variables as being a fixed quantity in a specific *run* of the program, i.e. when thinking locally about what happens in a specific line of code or specific operation. But there is also the global perspective of considering all possible values a random variable can take, i.e. the distribution over its values. This is perspective the logic offers. When thinking of defining variables in Lean, both the language and the logic, the types are different in the two cases! Say in the language, a variable has type A, then in the logic, it is given type $RV\,A$ for a type constructor RV (random variable). This causes technical challenges in defining and connecting the language and logic in lean and particularly in the instantiating in Iris, since there is no previous instantiation (as far as I am aware) of the variables not being the same across language and logic.

Note that "global" reasoning tends to conflate the variables does not mean non-compositional still enjoy a form of compositional reasoning using the logic, which is the independence of random variables. Thus, the compositionality offered by the semantics of the language, the line-by-line compositionality since each term/line is given a denotation which composes cleanly, may be understood as horizontal compositionality. The logic on the other hand, allows us to simultaneously talk about the variables introduced in every line of the program through the sequencing (monadic bind) of programs. However, the structure of assertions implicitly inform us of the independence between the variables where present, which can be understood as a *vertical* compositionality.

`<insert: revise the above into talking about this compositionality stuff elsewhere>`

3.10.1 Substitution inconsistency

Lilac

3.11 Equality of dependently typed records in Lean

3.12 Choice of Kripke Resource Monoid (KRM) over Unital ordered Resource algebra (ORA)

ORA's are established as the standard by former Rocq-iris projects, but I couldn't see why not just directly use the KRM. It was conceptual simplicity traded for some possibly unweildy proofs. The unweildiness turned out to be restricted to a single file?

3.13 Deep vs shallow embedding

3.14 Support for recursion over the data structure even with (shallow?/deep? see SSProve talk) embedding

(compare with SProp usage in Rocq for ORA? or something...) Even though the monotonicity property did not require induction, substitution atleast as defined in the paper requires it.

The aim, is to do things as given in the paper, and then think about if substitution could be done away with? If removing the ability to do recursion over assertions is desirable, etc

3.15 Strong abstractions across the proof development

3.15.1 The language

This is built on the existing abstractions. We use the typeclass `DenotationalMeas` to decouple the syntax from the language from the logic. The language only needs to me able to interpreted as measurable maps with a few additional requirements as given by `DenotationalMeas`.

3.15.2 Resource monoid

for the version of Lilac with/without conditioning

3.15.3 Separate treatment of BI connectives and Probability specific connectives

3.16 Metaprogramming and tactical support

Bibliography

Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017. doi: 10.18637/jss.v076.i01. URL https://www.jstatsoft.org/index.php/jss/article/view/v076i01.

Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*, pages 62–79. Springer, 2016. doi: 10.1007/978-3-319-29604-3_5. URL http://dx.doi.org/10.1007/978-3-319-29604-3_5.

Frank Wood, Jan Willem van de Meent, and Vikash Mansinghka. A new approach to probabilistic programming inference. In *Proceedings of the 17th International conference on Artificial Intelligence and Statistics*, pages 1024–1032, 2014.

Sam Staton. Commutative semantics for probabilistic programming. In *Programming Languages and Systems: 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*, page 855–879, Berlin, Heidelberg, 2017. Springer-Verlag. ISBN 978-3-662-54433-4. doi: 10.1007/978-3-662-54434-1_32. URL https://doi.org/10.1007/978-3-662-54434-1_32.

Jialu Bao, Emanuele D'Osualdo, and Azadeh Farzan. Bluebell: An alliance of relational lifting and independence for probabilistic reasoning. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704894. URL https://doi.org/10.1145/3704894.

Janine Lohse, Tim Rohde, Jimmy Xin, Niklas Mück, Iona Kuhn, Derek Dreyer, Deepak Garg, and Emanuele D'Osualdo. First steps towards probabilistic iris: Harmonizing independence, conditioning, and dynamic heap allocation, 2026. URL https://arxiv.org/abs/2605.13765.

Rémy Degenne. Markov kernels in mathlib's probability library, 2025. URL https://arxiv.org/abs/2510.04070.

Chris Heunen, Ohad Kammar, Sam Staton, and Hongseok Yang. A convenient category for higher-order probability theory. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, page 1–12. IEEE, June 2017. doi: 10.1109/lics.2017.8005137. URL http://dx.doi.org/10.1109/LICS.2017.8005137.

Michikazu Hirata, Yasuhiko Minamide, and Tetsuya Sato. Semantic Foundations of Higher-Order Probabilistic Programs in Isabelle/HOL. In Adam Naumowicz and René Thiemann, editors, *14th International Conference on Interactive Theorem Proving (ITP 2023)*, volume 268 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-284-6. doi: 10.4230/LIPIcs.ITP.2023.18. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2023.18.

Reynald Affeldt, Cyril Cohen, and Ayumu Saito. Semantics of probabilistic programs using s-finite kernels in dependent type theory. *ACM Trans. Probab. Mach. Learn.*, 1(3), August 2025. doi: 10.1145/3732291. URL https://doi.org/10.1145/3732291.

Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*. 2024. URL https://leanprover.github.io/theorem_proving_in_lean4/. with contributions from the Lean Community.

Mario Carneiro. The type theory of lean. Master's thesis, Carnegie Mellon University, 2019. URL https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf.

Sheldon Axler. *Measures*, pages 13–72. Springer International Publishing, Cham, 2020. ISBN 978-3-030-33143-6. doi: 10.1007/978-3-030-33143-6_2. URL https://doi.org/10.1007/978-3-030-33143-6_2.

John M. Li, Amal Ahmed, and Steven Holtzen. Lilac: A modal separation logic for conditional probability. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591226. URL https://doi.org/10.1145/3591226.

Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, pages 68–85, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg. ISBN 978-3-540-39041-1.

Peter O'Hearn, John Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In Laurent Fribourg, editor, *Computer Science Logic*, pages 1–19, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. ISBN 978-3-540-44802-0.

Cristiano Calcagno, Peter W. O'Hearn, and Hongseok Yang. Local action and abstract separation logic. In *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*, pages 366–378, 2007. doi: 10.1109/LICS.2007.30.

David J. Pym, Peter W. O'Hearn, and Hongseok Yang. Possible worlds and resources: the semantics of bi. *Theoretical Computer Science*, 315(1):257–305, 2004. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2003.11.020. URL https://www.sciencedirect.com/science/article/pii/S0304397503006248. Mathematical Foundations of Programming Semantics.

Stephen Brookes and Peter W. O'Hearn. Concurrent separation logic. *ACM SIGLOG News*, 3 (3):47–65, August 2016. doi: 10.1145/2984450.2984457. URL https://doi.org/10.1145/2984450.2984457.

D. Galmiche, D. Méry, and D. Pym. The semantics of bi and resource tableaux. *Mathematical Structures in Computer Science*, 15(6):1033–1088, 2005. doi: 10.1017/S0960129505004858.

Peter W. O'Hearn and David J. Pym. The logic of bunched implications. *Bulletin of Symbolic Logic*, 5(2):215–244, 1999. doi: 10.2307/421090.

Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. Mosel: a general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236772. URL https://doi.org/10.1145/3236772.

Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. 11 2014.

Thierry Coquand. An analysis of girard's paradox. 10 1999.

leanprover community. Metaprogramming in lean 4. https://leanprover-community.github.io/lean4-metaprogramming-book/, 2024. Online book.

Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. ISBN 0262026651.

Gilles Barthe, Justin Hsu, and Kevin Liao. A probabilistic separation logic. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–30, December 2019. ISSN 2475-1421. doi: 10.1145/3371123. URL http://dx.doi.org/10.1145/3371123.

Shing Hin Ho, Nicolas Wu, and Azalea Raad. Bayesian separation logic, 2025. URL https://arxiv.org/abs/2507.15530.

Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. Formal certification of code-based cryptographic proofs. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '09, page 90–101, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605583792. doi: 10.1145/1480881.1480894. URL https://doi.org/10.1145/1480881.1480894.

Ralf Jung, Robbert Krebbers, Jacques-henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi: 10.1017/S0956796818000151.

Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. *SIGPLAN Not.*, 52(1):205–217, January 2017. ISSN 0362-1340. doi: 10.1145/3093333.3009855. URL https://doi.org/10.1145/3093333.3009855.

Lars König. An improved interface for interactive proofs in separation logic, October 2022.

leanprover-community. iris-lean: Lean 4 port of iris, a higher-order concurrent separation logic framework. https://github.com/leanprover-community/iris-lean/, 2026. Accessed: 18th Jan 2026.